

Custom SQL Error Handler

=====

SQL Server Stored Procedure Example:

```
USE [RogStock]
GO
/***** Object:  StoredProcedure [dbo].[SP_TestErrorHandling]
Script Date: 04/12/2025 11:47:00 *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
-- =====
-- Author:          Roger Williams
-- Create date: 02/07/2025
-- Description: Returns a custom error number
-- =====
ALTER PROCEDURE [dbo].[SP_TestErrorHandling]
    @ErrorCustom int OUTPUT
AS
BEGIN
    -- SET NOCOUNT ON added to prevent extra result sets from
    -- interfering with SELECT statements.
    SET NOCOUNT ON;

    SELECT * FROM Login WHERE LOG_User = N'bob';

    IF (@@ROWCOUNT <> 0)
    BEGIN
        SET @ErrorCustom = 0;
    END
    ELSE
    BEGIN
        SET @ErrorCustom = 1000;
    END
END
```

Error List File Example:

```
1000,Login Not Found
1001,User Not Found Or Password Incorrect
1003,Error Creating LOT Record!
1004,Error Reading User Password
```

Visual Studio

```
//SQL server custom error handling
string CNST_STR_ERRORFILEPATH = string.Empty;
Dictionary<int, string> dicSQLErrors = new Dictionary<int, string>();

//resource files locations
readonly string CNST_STR_SQLCUSTOMERRORSPATH =
@"C:\Project\Resources\Errorlist.res";
```

NOTE: could have above file as list and read from it into the dictionary at startup!

e.g.:

```
public static Dictionary<int, string> dicSQLErrors = new
Dictionary<int, string>()
{
    { 1000, "Record Not Found!"},
    { 1001, "Error Saving Data!"},
    { 1003, "Error Creating Record!"},
    { 1004, "Error deleting Record!"}
};
```

```
public static bool InitCustomErrorHandler(string strPath)
{
    /*
    Created 02/07/2025 By Roger Williams

    Inits custom SQL error resource file path variable
    Then loads it into dictionary: dicSQLErrors

    Checks if passed path is null or file does not exist

    VAR

    strpath      - location of resource file

    RETURNS

    true if ok

    */

    StreamReader strmTemp = null;
    string strTemp = string.Empty; ;
    string strError = string.Empty; ;
    string strMsg = string.Empty; ;

    if (strPath.Length == 0 || !File.Exists(strPath))
    {
        return false;
    }
}
```

```

    }

    CNST_STR_ERRORFILEPATH = strPath;

    strmTemp = new StreamReader(CNST_STR_ERRORFILEPATH);

    while (!strmTemp.EndOfStream)
    {
        strTemp = strmTemp.ReadLine();
        //split into error number and error message

        //purposely add,
        strError = strTemp.Substring(0, strTemp.IndexOf(","))
+ 1);

        strMsg = strTemp.Remove(0, strError.Length);
        //remove it
        strError = strError.Remove(strError.Length - 1);

        dicSQLErrors.Add(Convert.ToInt32(strError), strMsg);
    }

    strmTemp.Close();
    strmTemp.Dispose();
    return true;
}

In SaveRecord():

        //create new STOCK_LOT record using stored procedure
then get @@IDENTITY -> new Lot nbr
        SQLCmd.CommandType =
CommandType.StoredProcedure;
        SQLCmd.CommandText = "sp_CreateNewLot";
        //@itemID varchar(50), @qty int, @LotNbr

int output
        SQLCmd.Parameters.Add("@itemID",
SqlDbType.VarChar, 50).Value = this.CMBSTKI_ItemID.Text;
        SQLCmd.Parameters.Add("@Location",
SqlDbType.VarChar, 30).Value =
this.DGVLocations.Rows[intSelectedLoc].Cells[0].Value;    //
DTRTemp["Location"];

        SQLCmd.Parameters.Add("@qty",
SqlDbType.Int, 0).Value =
Convert.ToInt32(this.DGVLocations.Rows[intSelectedLoc].Cells[1].Value
);    //DTRTemp["Qty"];

        SQLCmd.Parameters.Add("@LotNbr",
SqlDbType.Int).Direction = ParameterDirection.Output;
        SQLCmd.Parameters.Add("@ErrorCustom",
SqlDbType.Int).Direction = ParameterDirection.Output;
        SQLCmd.ExecuteNonQuery();

```

```

        if
        (Convert.ToInt32(SQLCmd.Parameters["@ErrorCustom"].Value) == 0)
        {
            //create TRN Lot
            SQLCmd.CommandType =
CommandType.Text;
            SQLCmd.CommandText = "INSERT INTO " +
Modules.clsTables.CNST_STR_LOTTRN + " (LotT_ItemID, LotT_Nbr,
LotT_Qty, LotT_Location, LotT_Operation) " +
"VALUES ('" +
this.CMBSTKI_ItemID.Text + "'," + SQLCmd.Parameters["@LotNbr"].Value
+ "," + this.DGVLocations.Rows[intSelectedLoc].Cells[1].Value + "," +
+ this.DGVLocations.Rows[intSelectedLoc].Cells[0].Value + "'," +
Modules.clsData.CNST_STR_OPERATION_LOTCREATED + "');";

            SQLCmd.ExecuteNonQuery();
            blnLotCreated = true;
        }
        else
        {
            //show error to user

Modules.clsData.dicSQLErrors.TryGetValue(Convert.ToInt32(SQLCmd.Param
eters["@ErrorCustom"].Value), out strTemp);
            strTemp =
SQLCmd.Parameters["@ErrorCustom"].Value.ToString() + "\n\n" +
strTemp;

            MessageBox.Show("Error: \n\n" +
strTemp);
        }
    }
}

```